

## ITA06 - Machine Learning

### Exp 15. To classify Iris flowers using the Naïve Bayes algorithm.

#### Procedure

1. Import the Iris dataset and GaussianNB classifier.
2. Load the Iris dataset.
3. Train the Naïve Bayes model using the training data.
4. Predict the flower class.
5. Display the predicted result.

#### Dataset 1: Fruit Classification

| Weight (g) | Diameter (cm) | Sweetness (%) | Fruit  |
|------------|---------------|---------------|--------|
| 150        | 7.5           | 90            | Apple  |
| 160        | 7.8           | 88            | Apple  |
| 145        | 7.3           | 91            | Apple  |
| 120        | 5.5           | 70            | Orange |
| 125        | 5.8           | 72            | Orange |
| 130        | 6.0           | 74            | Orange |
| 155        | 7.6           | 89            | Apple  |
| 118        | 5.4           | 69            | Orange |
| 148        | 7.4           | 92            | Apple  |
| 122        | 5.7           | 71            | Orange |

```
[1] X = np.array([
    [150,7.5,90],
    [160,7.8,88],
    [145,7.3,91],
    [120,5.5,70],
    [125,5.8,72],
    [130,6.0,74],
    [155,7.6,89],
    [118,5.4,69],
    [148,7.4,92],
    [122,5.7,71]
])

y = np.array([
    "Apple", "Apple", "Apple",
    "Orange", "Orange", "Orange",
    "Apple", "Orange", "Apple", "Orange"
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

new_sample = np.array([[152,7.5,90]])
prediction = model.predict(new_sample)

print("Predicted Fruit:", prediction[0])
```

... Predicted Fruit: Apple

## Dataset 2: Student Placement Classification

### CGPA Aptitude Score Communication Score Placement

|     |    |    |            |
|-----|----|----|------------|
| 9.2 | 90 | 88 | Placed     |
| 8.8 | 85 | 82 | Placed     |
| 8.5 | 80 | 79 | Placed     |
| 7.2 | 72 | 70 | Not Placed |
| 6.8 | 68 | 65 | Not Placed |
| 9.0 | 92 | 90 | Placed     |
| 7.5 | 75 | 74 | Not Placed |
| 8.6 | 84 | 81 | Placed     |
| 6.5 | 60 | 58 | Not Placed |
| 8.9 | 88 | 86 | Placed     |

```

[8.8,85,82],
[8.5,80,79],
[7.2,72,70],
[6.8,68,65],
[9.0,92,90],
[7.5,75,74],
[8.6,84,81],
[6.5,60,58],
[8.9,88,86]
])

y = np.array([
    "Placed", "Placed", "Placed",
    "Not Placed", "Not Placed",
    "Placed", "Not Placed",
    "Placed", "Not Placed",
    "Placed"
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

new_sample = np.array([[8.7,86,84]])
prediction = model.predict(new_sample)

print("Predicted Placement:", prediction[0])
... Predicted Placement: Placed
```

## Dataset 3: Disease Diagnosis

| Temperature (°C) | Heart Rate (bpm) | Oxygen Level (%) | Disease  |
|------------------|------------------|------------------|----------|
| 39.0             | 110              | 94               | Positive |
| 38.5             | 108              | 95               | Positive |
| 37.0             | 78               | 99               | Negative |
| 39.2             | 115              | 93               | Positive |
| 36.8             | 75               | 98               | Negative |
| 38.8             | 112              | 94               | Positive |
| 37.2             | 80               | 98               | Negative |
| 39.1             | 114              | 92               | Positive |
| 36.7             | 74               | 99               | Negative |
| 38.9             | 111              | 93               | Positive |

```
^ - np.array([
    [39.0,110,94],
    [38.5,108,95],
    [37.0,78,99],
    [39.2,115,93],
    [36.8,75,98],
    [38.8,112,94],
    [37.2,80,98],
    [39.1,114,92],
    [36.7,74,99],
    [38.9,111,93]
])

y = np.array([
    "Positive", "Positive", "Negative",
    "Positive", "Negative", "Positive",
    "Negative", "Positive", "Negative",
    "Positive"
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

new_sample = np.array([[38.7,109,94]])
prediction = model.predict(new_sample)

print("Predicted Disease Status:", prediction[0])

... Predicted Disease Status: Positive
```

## Dataset 4: Employee Promotion Prediction

| Experience (Years) | Performance Score | Training Hours | Promotion    |
|--------------------|-------------------|----------------|--------------|
| 10                 | 95                | 50             | Promoted     |
| 8                  | 90                | 45             | Promoted     |
| 7                  | 88                | 40             | Promoted     |
| 3                  | 65                | 20             | Not Promoted |
| 2                  | 60                | 18             | Not Promoted |
| 9                  | 93                | 48             | Promoted     |
| 4                  | 70                | 25             | Not Promoted |
| 8                  | 89                | 42             | Promoted     |
| 2                  | 58                | 15             | Not Promoted |
| 6                  | 85                | 38             | Promoted     |

```
J  
Os  
[8,90,45],  
[7,88,40],  
[3,65,20],  
[2,60,18],  
[9,93,48],  
[4,70,25],  
[8,89,42],  
[2,58,15],  
[6,85,38]  
]  
  
y = np.array([  
    "Promoted", "Promoted", "Promoted",  
    "Not Promoted", "Not Promoted",  
    "Promoted", "Not Promoted",  
    "Promoted", "Not Promoted",  
    "Promoted"  
)  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)  
  
model = GaussianNB()  
model.fit(X_train, y_train)  
  
new_sample = np.array([[7,90,44]])  
prediction = model.predict(new_sample)  
  
print("Predicted Promotion Status:", prediction[0])  
  
... Predicted Promotion Status: Promoted
```

## Dataset 5: Vehicle Classification

**Engine Capacity (cc) Weight (kg) Mileage (km/L) Vehicle Type**

|      |      |    |      |
|------|------|----|------|
| 100  | 95   | 70 | Bike |
| 125  | 110  | 65 | Bike |
| 150  | 125  | 55 | Bike |
| 800  | 850  | 22 | Car  |
| 1000 | 950  | 20 | Car  |
| 1200 | 1050 | 18 | Car  |
| 110  | 100  | 68 | Bike |
| 900  | 900  | 21 | Car  |
| 140  | 120  | 58 | Bike |
| 1300 | 1100 | 17 | Car  |



```
[100,95,70],
[125,110,65],
[150,125,55],
[800,850,22],
[1000,950,20],
[1200,1050,18],
[110,100,68],
[900,900,21],
[140,120,58],
[1300,1100,17]
])

y = np.array([
    "Bike","Bike","Bike",
    "Car","Car","Car",
    "Bike","Car","Bike","Car"
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

new_sample = np.array([[135,118,60]])
prediction = model.predict(new_sample)

print("Predicted Vehicle Type:", prediction[0])
```

... Predicted Vehicle Type: Bike